

JomCode Coding Bootcamp

Type hints, classes and composition

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Read Python annotations without treating them as automatic runtime validation.
- 02 Model a small record using a dataclass.
- 03 Use composition and explicit dependencies instead of an inheritance framework.

Plain meaning → technical view → use

PLAIN MEANING

Describe the shape of data and group related behavior.

TECHNICAL VIEW

Type hints document intent; dataclasses and composition structure Python objects.

WHEN YOU USE IT

Pass a database dependency to a service without building a large inheritance tree.

TIP

Type hints aid understanding but do not validate runtime input by themselves.

Worked example 1: Annotated function



chapter_12_type_hints_classes_and_composition.py

```
1 def remaining(capacity: int, registered: int) → int:
2     return capacity - registered
3
4 print(remaining(20, 8))
5 print(remaining.__annotations__)
```

PREDICT Will an annotation automatically reject True?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

12

Annotations name capacity, registered and return as int.

WHY THIS HAPPENS

No. The application must validate incoming data at runtime.

Exercise 1: Annotated normalizer

BUILD

Add str input/return annotations to an email normalizer. Demonstrate a valid call.

CHECK

Spaced mixed-case email normalized. Explain that integer input is not automatically rejected by the annotation.

IF STUCK

Annotations support tooling. The method call still governs runtime behavior.

Worked example 2: Dataclass record



chapter_12_type_hints_classes_and_composition.py

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Workshop:
5     title: str
6     capacity: int
7
8 workshop = Workshop("Python", 20)
9 print(workshop.title, workshop.capacity)
```

PREDICT Does the dataclass validate capacity 0?

Example 2: output and reason

TERMINAL >_ EXPECTED OUTPUT

Python 20

WHY THIS HAPPENS

Not automatically. Use domain validation or Pydantic at the API boundary later.

Exercise 2: Validated record factory

BUILD

Create a Workshop dataclass and a function that validates capacity before constructing it.

CHECK

1 and 500 accepted. True/0/501 rejected.

IF STUCK

Validate the field before creating the instance.

Worked example 3: Injected dependency



chapter_12_type_hints_classes_and_composition.py

```
1 class Calculator:
2     def __init__(self, operation):
3         self.operation = operation
4
5     def apply(self, value):
6         return self.operation(value)
7
8     def double(value):
9         return value * 2
10
11 print(Calculator(double).apply(3))
```

PREDICT Which behavior can a caller replace?

Example 3: output and reason

TERMINAL	>_	EXPECTED OUTPUT
		6

WHY THIS HAPPENS

The injected operation supports a small controlled substitute in a test.

Exercise 3: Composition boundary

BUILD

Create a catalogue service receiving a list of workshop dictionaries. Return IDs without owning terminal input.

CHECK

Two records => [w1,w2]. Empty => []. Caller-supplied dependency used.

IF STUCK

Receive data explicitly so tests can supply a small dependency.

Checkpoint

PROVE IT

Explain annotations, runtime validation and instance state as separate concepts.